

MIPS 32-bits

Fase 2: Implementación Instrucciones Tipo I & J

Maestro: Jorge Ernesto Lopez Arce Delgado

Alumnos:

Luis Giovanni Rodríguez Hernández

Sara Iriel Alonso Jaquez

Jorge Abraham Moran Encina

Leonel Octavio Rios Loreto

Objetivo

Implementar un procesador MIPS de 32 bits utilizando Verilog para ejecutar instrucciones tipo R, tipo I y tipo J.

También se buscó:

- Implementar acceso a memoria
- Ejecutar instrucciones branch
- Crear archivo de instrucciones .txt
- Validar el funcionamiento mediante simulación

Modulos Desarrollados

- Principales módulos:
- PC
- ALUMIPS
- ALUCTRL
- UNITCTRL
- BR (Banco de registros)
- MEMI
- MEMA
- Mux2a1
- SignExtend
- DPTR
- Techbench

FUNCIONAMIENTO DEL DATAPATH

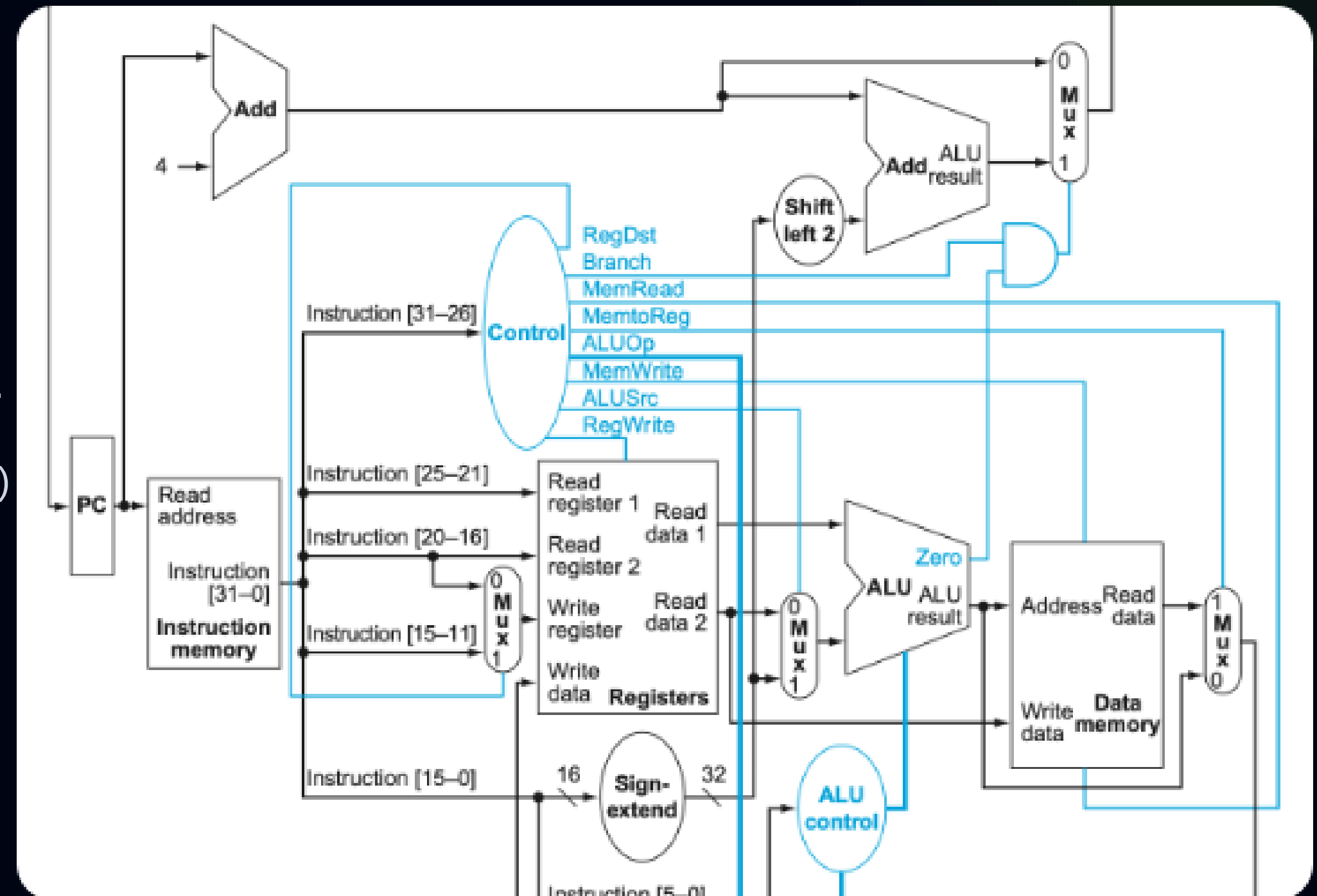
Flujo de ejecución:

1. El PC obtiene la instrucción
2. MEMI entrega la instrucción
3. UNITCTRL genera señales
4. BR entrega operandos
5. La ALU ejecuta operaciones
6. MEMA accede a memoria
7. El resultado regresa al banco de registros

Arquitectura

El sistema ejecuta una instrucción completa en un solo ciclo de reloj. Se han integrado componentes críticos:

- ✓ **Unidad de Extensión:** Para manejar inmediatos de 16-bits.
- ✓ **Módulos de Memoria:** Separación de Instrucciones (MEMI) y Datos (MEMA).
- ✓ **Lógica de PC:** Selección dinámica entre PC+4, Branch y Jump.



Instrucciones Soportadas

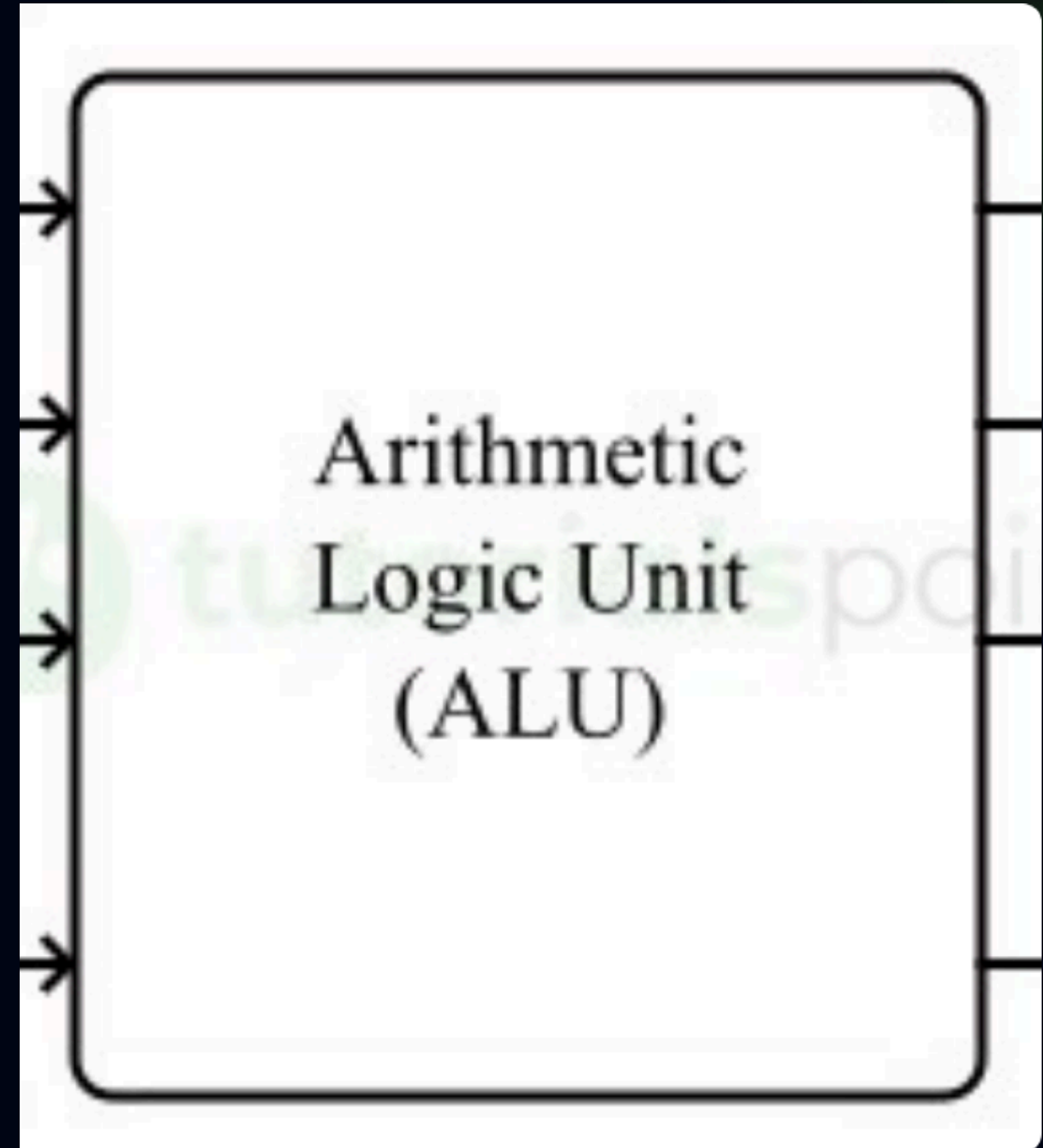
Categoría	Instrucciones Agregadas	Operación Principal
Tipo R	ADD, SUB, SLT, NOR	Aritmética y Lógica entre registros
Tipo I (Memoria)	LW (Load), SW (Store)	Transferencia entre RAM y Registros
Tipo I (Lógica)	ADDI, ORI, ANDI, SLTI	Operaciones con valores inmediatos
Control (I & J)	BEQ (Branch), J (Jump)	Cambio del flujo del programa

| Manejo de Memoria

LW & SW

La implementación de LW (Load Word) permite traer datos desde la memoria de datos (MEMA) al banco de registros.

Para esto, el datapath activa las señales MemRead y MemToReg, utilizando la ALU para calcular la dirección base más el desplazamiento (offset).



| Control de Flujo: Saltos

BEQ (Branch)

El salto condicional compara dos registros mediante una resta en la ALU. Si el resultado es cero (Zero flag), el PC se actualiza a la dirección de destino calculada por el sumador de branch.

J (Jump)

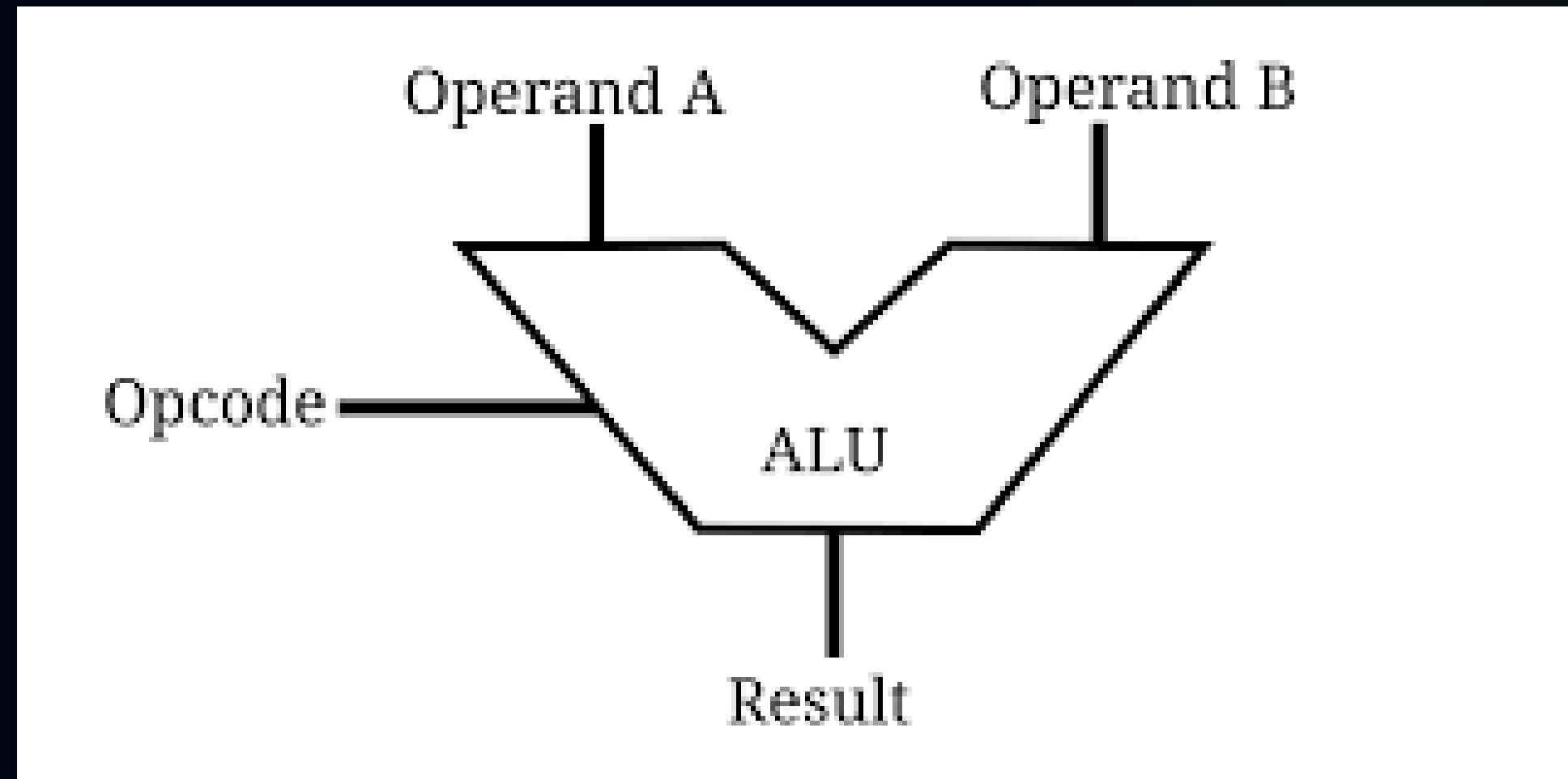
Salto incondicional que modifica los 26 bits inferiores del PC de forma directa. Se implementó un multiplexor 4:1 (Mux4a1) para priorizar estas transiciones de dirección.

ALU Y CONTROL

La ALU realiza operaciones aritméticas y lógicas dependiendo de las señales generadas por ALUCTRL y UNITCTRL.

Operaciones implementadas:

- Suma
- Resta
- AND
- OR
- NOR
- Comparación SLT



MEMORIA Y ARCHIVO DE INSTRUCCIONES

La memoria de instrucciones utiliza el archivo inst.txt.

```
$readmemb("\inst.txt", MEM);
```

Función:

- Cargar instrucciones binarias
- Simular ejecución del programa
- Ejecutar código ensamblador convertido a binario

